

Data Structures – Unit 09

Bucharest University of Economic Studies

Trees and Binary Trees



Trees - Definition

A nonlinear structure composed by a set of nodes in which one is called root, R , and a set of subtrees, $T_1, T_2, \dots, T_k$, each one connected by an edge to its corresponding root.



Trees - Characteristics

Each subtree has its own root.

Each root has its own set of children.

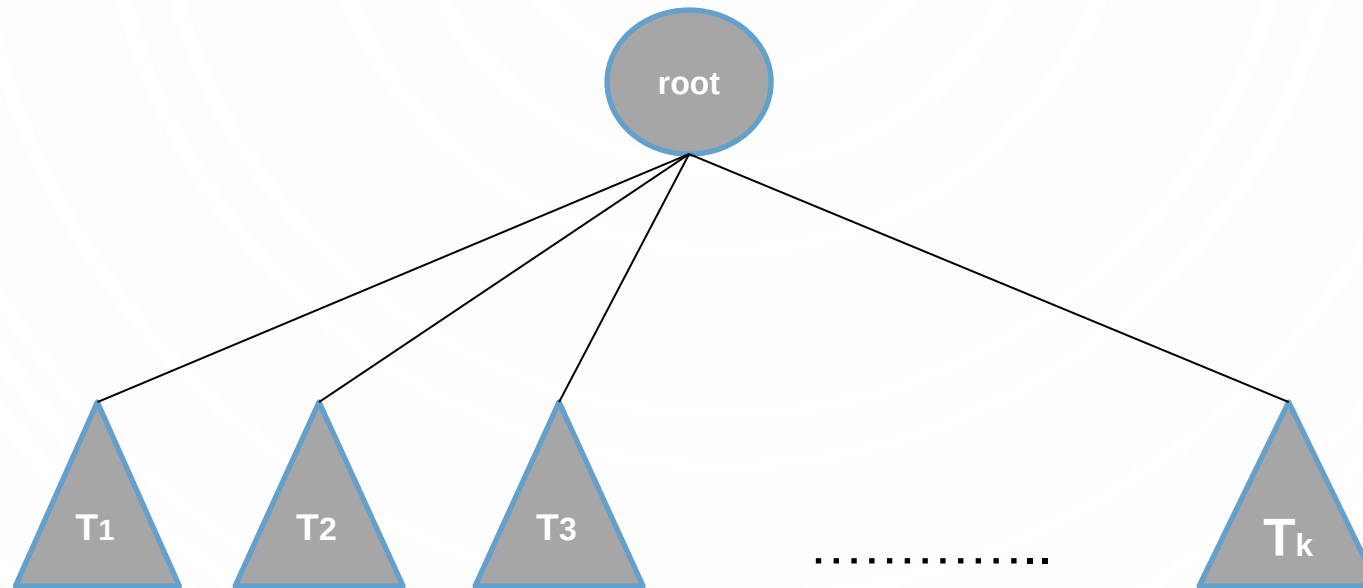
Each child has a node called parent, excepting the root node, at the top, which has no parent.

A tree with N nodes has $N-1$ edges or branches;

Nodes without children are called leaves or end nodes;



Trees - Characteristics



Trees - Characteristics

- A graph structure without cycles, connected and oriented;
- A single root node (the entry point in the structure);
- Efficient representations: Left child-Right sibling using the HEAP memory;
- Tree traversal or search: pre/post and in-order;
- Special types of trees: binary search trees, balanced trees, structural trees, B-trees, Red and Black trees;



Trees - Characteristics

A path from node n_1 to n_k is defined as the sequence of nodes $n_1, n_2, \dots, n_k$, in which every n_i node is the parent of the n_{i+1} node, for each $1 \leq i \leq k$.

The length of this path is the number of branches forming the path, which is $k-1$.

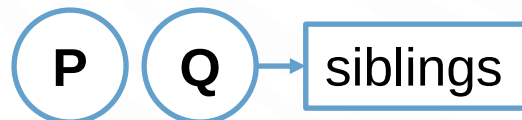
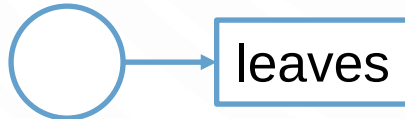
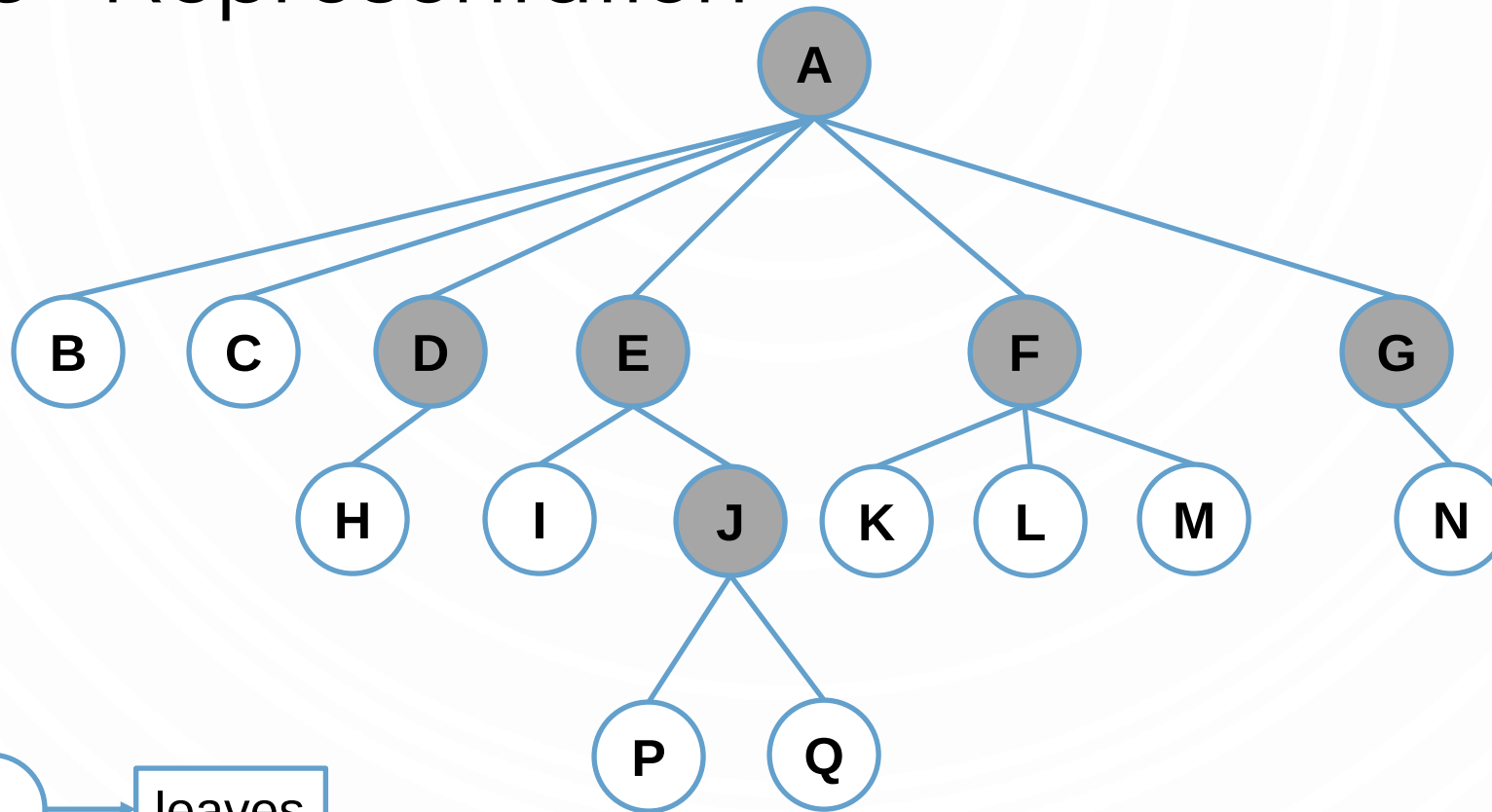


Trees - Characteristics

For each node n_i , its depth is defined as the length of the path from the root to that node.



Trees - Representation



Trees - Representation

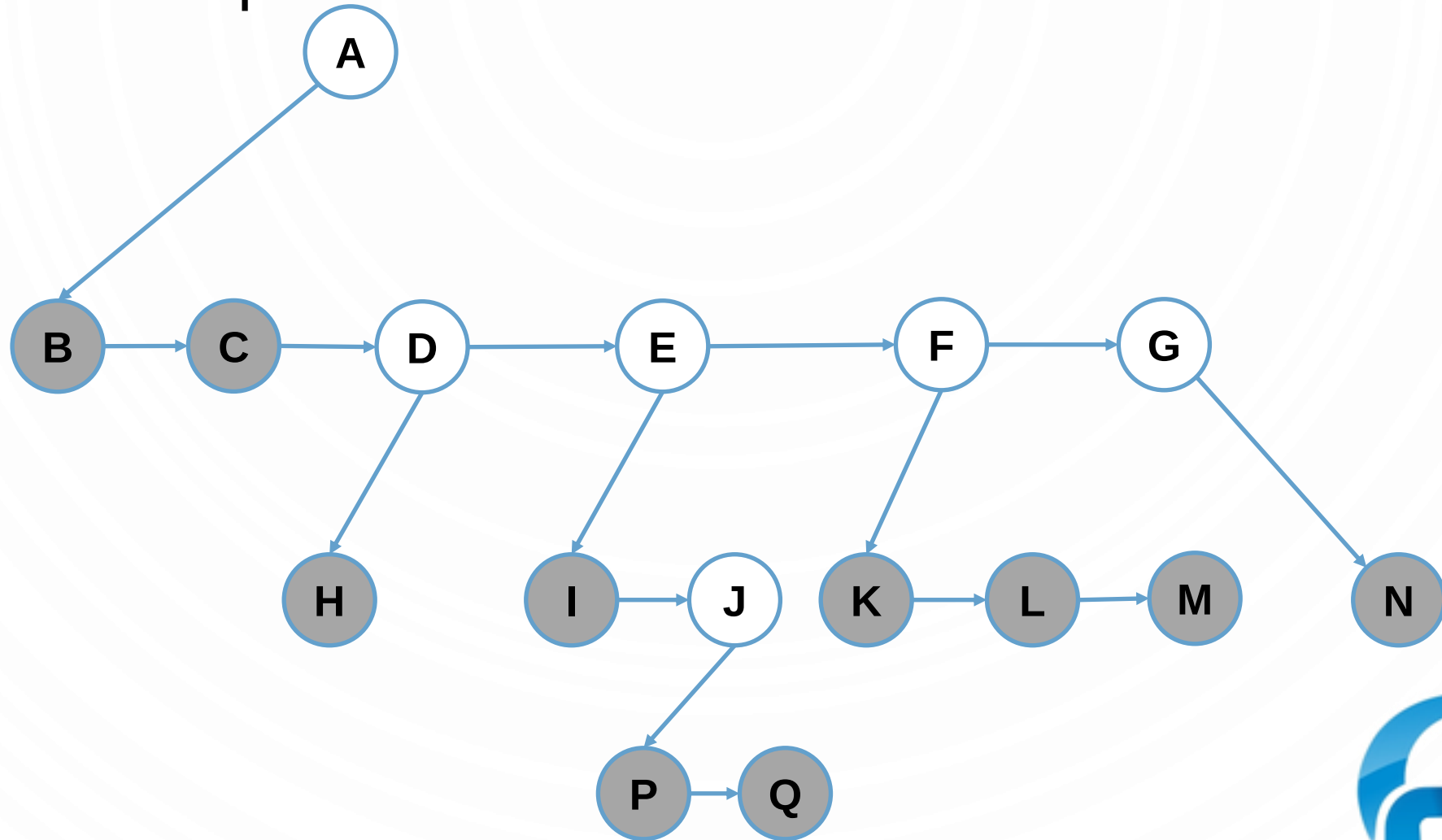
The left child – right sibling representation of a tree:

- the ID for the first descendant;
- the ID for the first sibling;
- the useful information;

```
struct TreeNode
{
    Object element;
    TreeNode* leftChild;
    TreeNode* rightSibling;
}
```



Trees - Representation



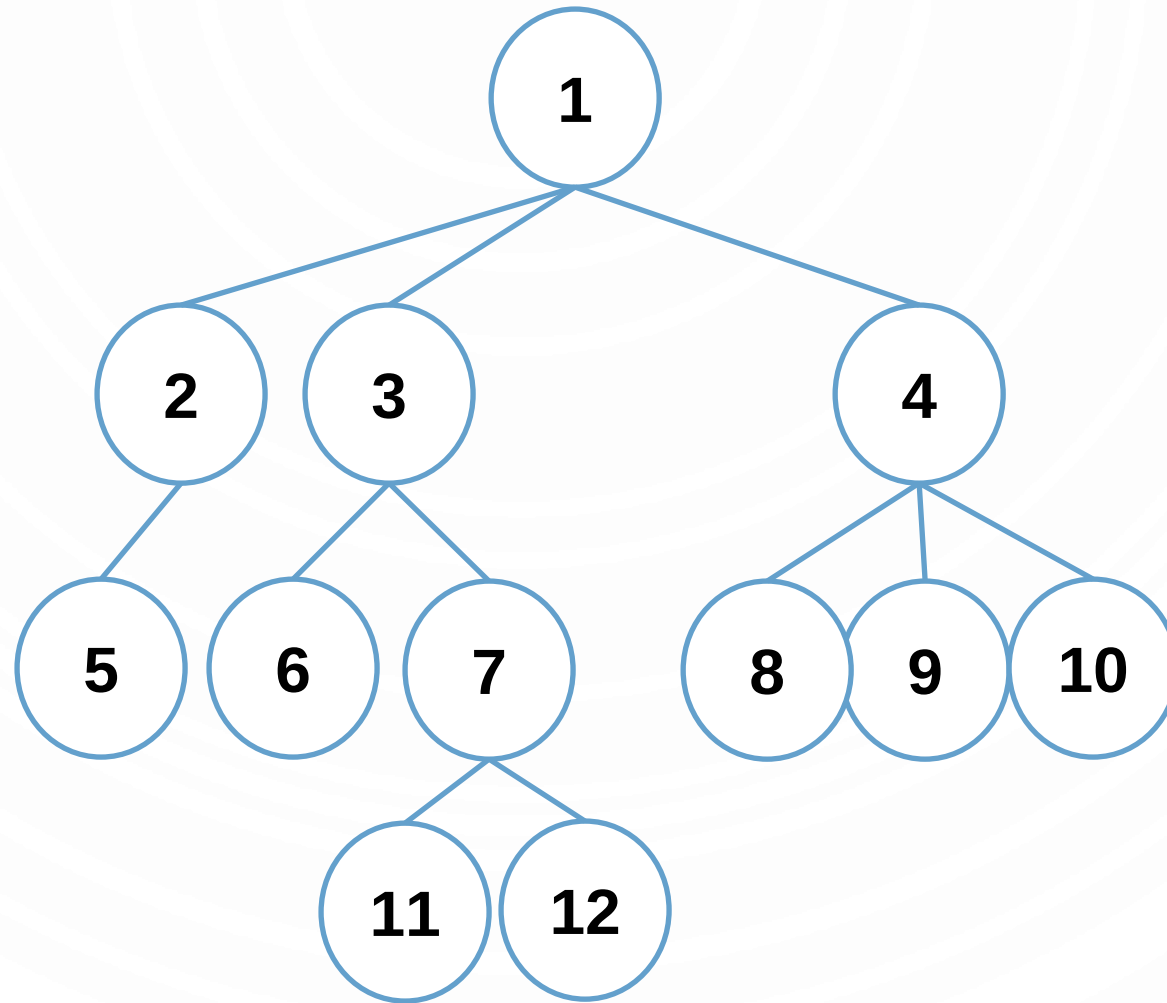
Trees - Representation

Fields for the data structure:

- The number of tree's nodes;
- The root node;
- A vector for the descendants;
- A vector for siblings;



Trees - Representation



Generic Trees - Representation

```
int root = 1, n = 12;
```

```
int child[] = {2, 5, 6, 8, 0, 0, 11, 0, 0, 0, 0, 0};
```

```
int sibling[] = {0, 3, 4, 0, 0, 7, 0, 9, 10, 0, 12, 0};
```

Using dynamic structures for tree representation:

- node ID;
- vector/list/or other linear structure with addresses of the descendants;



Trees - Representation

```
/* a generic tree with 10 descendants */  
struct nodArb{  
    int inf;  
    nodArb* parent;  
    nodArb* descendants[10];  
};  
  
/* a variable used to store a tree */  
nodArb* radAO=NULL;
```



Trees - Traversal

Tree traversal

Depth first (L-to-R or R-to-L)

Breadth
first

Pre-
order

In-order

Post-
order

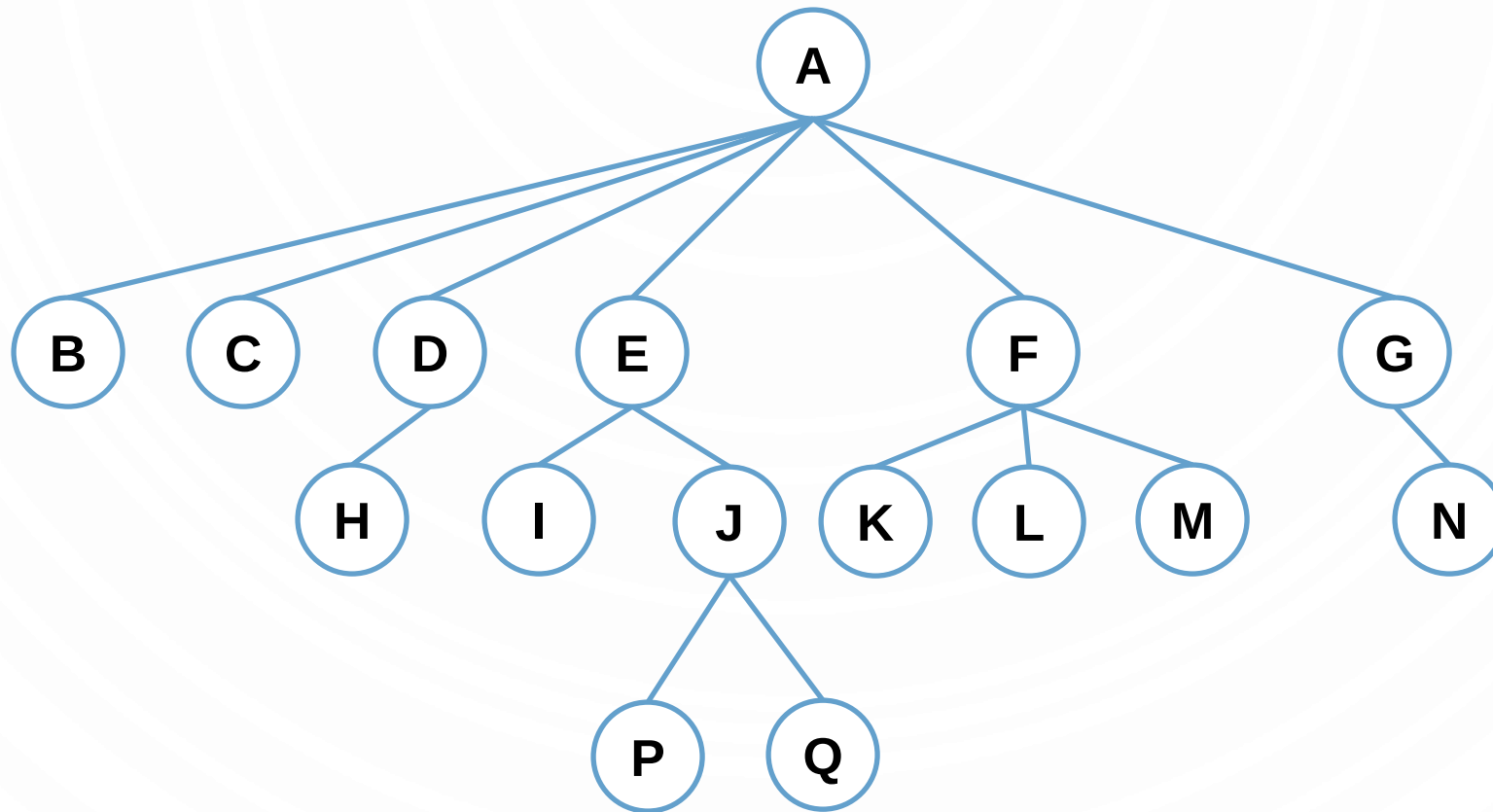


Depth first implementations

- Topological sorting
- Finding bridges in a graph
- Generating or solving mazes with one solution
- Finding strong connected graph components



Trees - Traversal



Trees - Traversal

Traversal of generic trees – recursive operation:

```
void listTree(TreeNode nod)
```

```
{
```

```
    1. processing the current node;
```

```
    2. are there any descendants?
```

```
        2.1 for each descendant do recursive call
```

```
            listTree(descendent)
```

```
}
```



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

